Lab 4, Act 2

How much monitoring is enough?

NRES 746

Answer key. Answers are in magenta. Simulation results come from one run; yours will differ slightly.

Picking up where we left off

In Act 1 you built and validated a simple log-scale regression tool: fit a straight line to the *log* counts, and the slope estimate comes out unbiased, with a spread that matched a formula you checked against your own simulation. Now we put that tool to work on the question the agency actually cares about: what does it take to run a monitoring program that reliably catches a real decline, without spending more than it has to?

The design problem

The true abundance in year 0 is $N_0 = 1000$ (assume it is known). Each animal present is detected independently with probability 0.02 per person-day of searching. A monitoring **bout** uses k person-days of total search effort (say, 3 observers for 1 day, or 1 observer for 3 days – see 1a for why these are treated the same), repeated every s years ($s = 1, 2, 3, 4$, or 5), starting in year 1, for as long as they fit within the 25-year window ($t = 1, 1 + s, 1 + 2s, \ldots \leq 25$). Costs are \$200 per person-day, plus \$2,000 to mount each field expedition, regardless of its length.

Each bout produces one count index, I_t. A decline is **detected** if a straight-line fit of $\log(I_t)$ on t (the same analysis as Act 1) gives a negative slope with a two-sided p-value at or below 0.05. The management goal is to have **at least a 75% chance of detecting a decline of 25% or more over 25 years.**

What is the cheapest design that meets the goal, and how sure are we?

This Act has a lot of code to write, so you are **encouraged to use an AI assistant throughout Part 2**. But you will build and test one small function at a time rather than asking for the whole thing at once, and you'll check each piece yourself before trusting it. Part 1 is still worth doing on paper first: resist the urge to open a laptop or an AI assistant until you've thought each piece through.

Part 1: Plan the pieces (individual, ~18 min)

We're going to build the simulation out of four small functions, one piece of the problem at a time. Work out what each piece should do before you write any code.

1a. A single animal is detected during a bout if *at least one* of the k independent person-days of search finds it, each with probability 0.02. What is the probability it is **not** detected on a single person-day? Use that to write a formula for $q(k)$, the probability it **is** detected somewhere during the whole bout.

Not detected on one person-day: $1 - 0.02 = 0.98$; on all k: 0.98^k. So $q(k) = 1 - 0.98^k$ ($q(3) \approx 0.0588$).

Check your formula: $q(3)$ should come out to about 0.059.

Notice $q(k)$ depends only on the *total* person-days k, not on how they're split between observers and days - 3 observers for 1 day and 1 observer for 3 days give the same q. What might make those two designs differ in the real world, even though the formula treats them identically?

E.g. one observer may learn the site over several days; weather or animal activity varies day to day, so spreading effort hedges against one bad day; animals may react to repeated visits. Any of these breaks the "independent, identical 2% trials" assumption.

1b. Using the same model as Act 1 ($\lambda = 0.75^{1/25}$, so a 25% decline compounds smoothly over 25 years), write the formula for the true expected abundance at year t, starting from $N_0 = 1000$.

$N_t = N_0\lambda^t = 1000\left(0.75^{1/25}\right)^t \quad (N_{25} = 750)$

Check: N_{25} should come out to a clean 750.

1c. At year t, the true abundance is N_t and each of those animals is detected independently with probability $q(k)$ (from 1a). What probability distribution would you use to generate the *number of animals actually counted* in that year's survey? Why that one?

Binomial: $I_t \sim \text{Binomial}(N_t, q(k))$. A fixed number of animals (N_t), each detected independently with the same probability, and we count the successes. (Poisson is acceptable as an approximation, since q is small.)

1d. Two quick predictions, before you build anything:

- Do you expect a **bigger bout** (larger k) or **more frequent surveys** (smaller s) to matter more for catching the decline?
- If the population were actually **stable** (no decline at all), would you expect this test to falsely detect a decline exactly 5% of the time? More often? Less often?

Open-ended for k vs. s (they turn out to matter by comparable amounts). False declines: less than 5%, about 2.5%, because the rule needs a negative slope *and* significance; half of the 5% significant results are increases.

Group check-in: finalize your plan (~5 min)

Compare your three pieces (1a-1c) and your two predictions (1d) with your group. Agree on one version of each before opening any laptops.

Laptops open now. From here on, Part 2 is a full group effort. Feel free to divide and conquer- not everyone needs to write out every answer: if one groupmate's packet has a result filled in, that's fine to stand for the whole group, and the rest of you can leave that spot blank. (The Reflection section at the very end is the one exception – answer that individually, in your own words.) You are encouraged to use an AI assistant throughout Part 2 - there's a lot of code to write today. Build and test one function at a time; don't move on to the next piece until the current one checks out. **Turn in one R script per group** with all the code you worked out together (most groups will build this with an AI assistant, and that's expected), in addition to your written packets – one member should email the script to the instructor, with everyone in your group listed on the email. [Note: I don't usually want to see your AI-generated code but this time is an exception!]

Part 2: Build, test, and search (laptops, AI encouraged, ~55 min)

For each piece below, build the function as a group (an AI assistant is a good way to get a working draft quickly), then run the check described and **record the result** in the space provided. You do not need to write any code by hand- your group's script covers that (see above).

2a. Detection probability. Build `detect_prob(k, pobs = 0.02)`, implementing your formula from 1a.

`detect_prob(3)` = 0.0588 (should be about 0.0588)

2b. Abundance trajectory. Build `true_abundance(t, N0 = 1000, lambda = 0.75^(1/25))`, implementing your formula from 1b. It needs to accept a whole vector of years at once, not just one at a time since that's how you'll call it in 2c and beyond.

`true_abundance(0)` = 1000 (should be 1000)

`true_abundance(c(0, 25))` = 1000, 750 (should return 1000 and 750 *together*, as one vector)

2c. Simulated counts. Build `simulate_counts(t_survey, k, N0 = 1000, lambda = 0.75^(1/25), pobs = 0.02)`, which returns one simulated count per year in `t_survey`. It should call your functions from 2a and 2b inside it rather than recomputing those formulas from scratch. Run it a few times with `t_survey = seq(1, 25, by = 1)` and `k = 3`, and check that the output is a **vector of 25 counts** (its `length()` should match `length(t_survey)`) – not a single recycled number, which is what you'd get if 2b's function silently failed on a vector of years.

Do the counts trend downward over time and stay well below the true abundance (detection is far from perfect)?

Yes: e.g. 65, 49, 47, . . . , 46, 42: noisy, trending down, and far below the true abundance (under 6% detected).

2d. Decision rule. Build `detect_decline(t_survey, counts, alpha = 0.05)`, implementing the decision rule from the design problem above (a negative slope *and* a p-value at or below `alpha`); it should return `TRUE` or `FALSE`.

Run `simulate_counts()` under the real decline ($\lambda = 0.75^{1/25}$) several times: how often did `detect_decline()` return `TRUE`? most of the time (about 84%) Run it with $\lambda = 1$ (no decline) several times: how often `TRUE`? rarely (about 2 to 3%)

2e. Power, and a validation check. Combine 2c and 2d into `estimate_power(k, s, R = 1000, lambda = 0.75^(1/25), alpha = 0.05)`: build the survey years from `s` (`seq(1, 25, by = s)`), run `R` replicates, and return the fraction where a decline was detected. Before trusting it, set $\lambda = 1$ and estimate the false-decline rate for $k = 3$, $s = 1$.

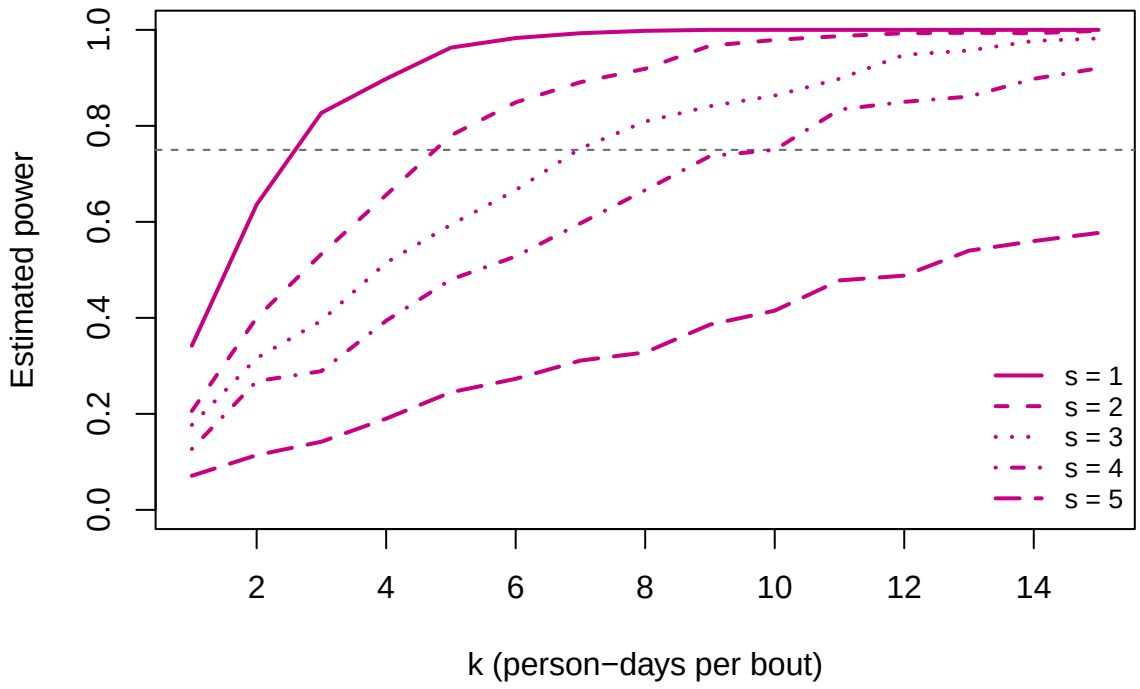
False-decline rate at $\lambda = 1$: 0.023 (Monte Carlo SE 0.003)

Does this roughly match your prediction in 1d?

Yes: about 2.5%, roughly half of 5%.

2f. The design space. Estimate the power for $k = 1, \ldots, 15$ and $s = 1, \ldots, 5$ (1,000 replicates per design is enough). Plot power against k, with one line per s and a horizontal line at 0.75. Describe the pattern. Did your 1d prediction (whether k or s mattered more) hold up?

Power rises with k for every s, and every curve shifts right as s grows: annual surveys pass 0.75 around $k = 3$; every-5-year surveys never do by $k = 15$. k and s matter by roughly comparable amounts.



2g. Cost and recommendation. Compute the cost of every design in your grid ($200 per person-day, $2,000 per expedition, however many bouts the design uses over 25 years). Among designs with estimated power at or above 0.75, find the cheapest few, and re-check your top 2-3 candidates with 5,000 replicates each (Monte Carlo error matters most right near a threshold).

k	Interval s	Cost ($)	Power (5,000 reps)	MC SE
10	4	28,000	0.757	0.006
11	4	29,400	0.805	0.006
7	3	30,600	0.738	0.006

Which design do you recommend, and why that one rather than the cheapest one that just squeaked past 0.75? (If you're curious whether this recommendation would change if the analysis were done on the raw counts instead of the log counts, that's what the optional Act 3 explores.)

$k = 11$, $s = 4$ ($29,400): it clears 0.75 with a margin of several Monte Carlo SEs. The cheapest design sits right at the threshold, so its true power could plausibly be below 0.75. (Recommending the cheapest is defensible if you re-checked it and say why you're comfortable with the margin.)

If you finish early: environmental noise. So far the only randomness in the abundance trajectory is sampling (detection) error. Real populations also experience year-to-year shocks that *persist*, such as a drought year that reduces the population, which the next year starts from. Modify `true_abundance()` so that $N_t = N_{t-1}\,\lambda\,e^{\varepsilon_t}$ with $\varepsilon_t \sim \text{Normal}(0, 0.05)$, and then rerun (i) the power of your recommended design and (ii) the false-decline rate when $\lambda = 1$. What happened, and which assumption from Act 1 is no longer true? What does this suggest about the kind of statistical model you would want for real monitoring data?

Power with environmental noise: about 0.50 False-decline rate when $\lambda = 1$: about 0.15

Power drops and false declines jump: persistent shocks make consecutive years' deviations correlated, so the independence assumption from Act 1 fails, and a run of bad years looks like a trend. Real monitoring data need a model that handles temporal autocorrelation (e.g. a state-space or time-series model).

Reflection (~8 min)

R1. Summarize your recommended design, its cost, and its estimated power. Which assumption of the simulation would you worry about most, and how would you find out whether it matters?

Open-ended (name the design, cost, power, and a specific assumption, e.g. independent detection, no environmental persistence, known N_0).

R2. In three or four sentences, explain your recommendation to the agency's director, who has no statistical training.

Open-ended (no jargon such as "power" or "p-value" without a plain-language gloss).

R3. Is the grid-search a reasonable approach for finding an optimal solution? Can you think of any better ways to optimize the experimental design in this case?

Reasonable here: only 75 designs, and k and s are naturally whole numbers, and the grid shows the whole power surface. Better: check every design cheaply, then re-run only those near 0.75 with many replicates; or use that power rises steadily with k (and falls with s) to search each row more cleverly. Grids stop being practical as more design variables are added.

R4. How did you use an AI assistant while building your four functions? What did you have to check, correct, or re-prompt for?

Open-ended.